AIQA — Business Logic & Architecture

Version 2.0 — Updated Hybrid Multi-Agent Architecture

Update basis: Original Business Logic v1.0 updated with approved v6.0 Hybrid Multi-Agent architecture. The original project-centric rules remain baseline: unified test generation, first-class RBAC, Security/Penetration, Edge/Business Logic, API latency, Playwright execution, selective reporting and local/API model abstraction. ■filecite■turn6file0■

New core rule: Central AIQA Orchestrator + specialized testing intelligence + Test Case Validator + common Execution Orchestrator/Workers + AI Model Gateway. Agents decide/analyze/validate; executors perform actual tests. ■filecite■turn6file1■

1. Product Vision

AIQA is a project-centric software quality platform that ingests project documentation and application context, generates structured tests across multiple dimensions, validates them, executes them, observes real-time behavior, preserves evidence and produces filtered/exportable reports. It is initially intended for standalone/personal use, with local AI preferred and API fallback when stronger reasoning is required. ■filecite■turn6file0■

2. Core Business Modules

Module	Business Rule
Dashboard	Project health, execution/pass-fail/test/report ratios, interactive pie/bar analytics and project selection.
Test Case Generator	Ingest FRD/SRD/PRD/Figma and project evidence; generate categorized positive/negative tests in one interactive view.
AIQA Orchestrator	Create plans, route work, prioritize by risk/change/test value/dependencies and coordinate specialized capabilities.
Test Case Validator	Verify traceability, quality, duplicates, coverage and risk before automatic execution.
Test Execution Hub	Schedule approved tests through DAG/queue/workers and stream status/evidence.
RBAC Testing	Fetch actual roles/users; validate positive/negative access; show active role/access path in real time.
API Testing	Validate request/response, schemas, errors, headers, latency, speed, timeouts and rate limits.
Mobile Testing	Select compatible Android/Samsung or iOS/Apple devices, lock resources and execute eligible non-UI/UX mobile coverage.
Custom Testing	Select project module/feature and define focused testing objective.
Feature Map / Flow	Expose features and understandable project/system call flow.
Reports	Bug/test/audit reports with full-project or module-only PDF/Excel export.
Settings / AI Model	Configure execution, reporting, integrations and local/API model routing. ■filecite■turn6file0■

3. Project Context & Quality Graph

• Create/select project; all requirements, features, tests, executions, findings, metrics and reports remain project-scoped.

• Project selection dynamically changes Dashboard, Generator, Execution, RBAC, API, Mobile, Custom Testing and Reports.

• Ingest FRD, SRD, PRD, Figma/design, API specs, repository metadata, DB schema, existing tests and historical defects when available.

• Maintain traceability across requirement → feature → module → API/DB/UI → test → execution → evidence → finding/report. Relational relationships are authoritative; vector retrieval is supplemental. ■filecite■turn5file8■

4. Hybrid Multi-Agent Business Logic — NEW

• One central AIQA Orchestrator instead of an independent model for every testing type.

• Requirement Intelligence: understands FRD/SRD/PRD/Figma/API/DB/repository context and feeds the Quality Graph.

• Test Design Intelligence: Functional, Non-Functional, UI/UX, Edge, Scenario, Business Logic and Regression.

• Security Intelligence: Security, Penetration, RBAC, authentication and authorization.

• API/Performance Intelligence: API contracts, latency, performance and load-oriented test design.

• Mobile Intelligence: platform/device-aware test selection and planning.

• Evidence/Result Intelligence: evidence correlation, failure classification, RCA hypotheses, flakiness and reporting.

• Orchestrator coordinates plan, routing, priorities, dependencies and aggregation. ■filecite■turn6file1■

5. Agent ≠ Executor

Agent: decides what/why/priority, generates and validates. **Executor:** performs browser/API/DB/security/mobile actions and collects evidence. Execution uses DAG scheduling, queues, worker dispatch and resource/device locks. Workers include Playwright/Chromium, API/HTTP, DB, Security/Penetration and Maestro/Appium. ■filecite■turn6file1■

6. Test Generation & Quality Gate

• Unified categories: Functional, Non-Functional, UI/UX, Edge, Scenario, Business Logic, Security, Penetration, RBAC, Regression, Database and API. ■filecite■turn6file0■

• Generate positive and negative cases with preconditions, steps, expected result, priority, category and traceability.

• Critical requirements must map to tests; duplicate/overlapping cases are detected.

• Edge/Business/RBAC/Security coverage is checked against risk.

• AI-generated tests pass configured quality/benchmark rules before unrestricted automatic execution. ■filecite■turn5file9■

7. RBAC — First-Class Business Logic

• Fetch actual roles/users before generation; build role × permission × action coverage.

• Generate allowed-action and unauthorized-action tests; validate privilege boundaries and segregation of duties.

• Show compact role tabs connected visually; active access path glows during execution.

• RBAC is project-scoped and authorization is enforced server-side. ■filecite■turn5file9■

8. Edge Case + Business Logic — First-Class

• Generate meaningful boundary, null/empty, malformed, min/max, unexpected-state, concurrency and recovery cases where applicable.

• Model business rules, calculations, dependencies, states and rule conflicts.

• Exercise conflicting/dependent rules, missing data, role changes and unexpected state transitions.

• Track edge coverage and trace business-logic tests to the requirement/rule that motivated them. ■filecite■turn5file9■

9. Security, API & Latency

• Security validates authentication, authorization, input/session/API controls; penetration is deeper authorized weakness discovery.

• Production security, destructive DB, production load and external side effects require explicit policy/authorization.

• API validates request/response bodies, headers, parameters, schemas, status, errors, timeouts and rate limits.

• Measure latency/response speed and useful p95/p99 percentiles; expose real-time API telemetry. ■filecite■turn6file0■turn5file3■

10. Execution & Real-Time

• Approved cases become schedulable tasks; DAG resolves dependencies.

• Independent tests continue after unrelated failures; dependent tests become BLOCKED.

• Distinguish product failures from environment/infrastructure failures.

• Support visible Playwright/Chromium execution and controlled parallel execution for the discussed ~1,000-test target.

• Capture screenshots, traces, logs, request/response and timing evidence.

• Normalize outcomes: QUEUED, RUNNING, PASSED, FAILED, BLOCKED, SKIPPED, CANCELLED, NEEDS_REVIEW and infrastructure/environment error.

• Events update the UI incrementally; reconnect reconciles authoritative state and processing is idempotent. ■filecite■turn6file1■

11. Mobile, Custom & Reporting

• Mobile: select compatible Samsung/Android or Apple/iOS device, check capability/health, lock device, fetch eligible project tests, execute non-UI/UX mobile coverage and stream evidence. ■filecite■turn6file1■

• Custom Testing: select a project module/feature and enter a focused objective; generate only targeted tests. ■filecite■turn6file0■

• Reports: Bug Report, Test Report and Final Audit Report; export full project or selected module in PDF/Excel. ■filecite■turn6file0■

12. AI Model Business Logic

• All agents use one AI Model Gateway; no provider is hard-coded.

• Local inference is preferred for standalone/personal use; API fallback is available for stronger reasoning.

• Routing can consider task type, context size, latency, availability and configured cost.

• Provider/model failure triggers controlled fallback or NEEDS_REVIEW, never fabricated output.

• 90%+ useful test-generation accuracy is a measured benchmark goal, not a guarantee; measure false positives/false negatives continuously. ■filecite■turn6file0■turn6file1■

13. End-to-End Business Flow

Project Create/Select → **Document & Context Ingestion** → **Requirement/Feature/Flow Understanding** → **Quality Graph** → **AIQA Orchestrator** → **Specialized Test Intelligence** → **Test Case Validator** → **Review/Selection** → **Execution Orchestrator** → **DAG/Queue** → **Specialized Executors** → **Real-Time Events** → **Evidence** → **Failure/RCA/Flakiness Intelligence** → **Dashboard** → **Bug/Test/Final Audit Reports** → **Full or Module Export.** This extends the original flow with the Orchestrator, validation and worker layers. ■filecite■turn6file0■turn6file1■

14. Primary Quality KPIs

KPI	Measure
Test Generation Accuracy	Correctness/relevance against reviewed benchmark cases.
Requirement Coverage	Requirements/features mapped to generated and executed tests.
Edge Coverage	Meaningful boundary/abnormal conditions covered.
Business Logic Coverage	Rules, calculations, dependencies, states and conflicts covered.
RBAC Coverage	Roles × permissions × critical actions × positive/negative scenarios.
Security Coverage	Security controls, negative cases and authorized penetration findings.
API Latency	Average/median and p95/p99 where applicable.
UI Stability	UI pass rate, interaction defects and browser stability.
Execution Reliability	Pass/fail/error/skipped distribution and flakiness rate.
Agent/Model Health	Model latency, fallback rate and validation rejection rate.
Execution Efficiency	Queue depth, worker utilization, task latency and event lag.

15. Recommended Architecture Rules

• Project isolation for every artifact/runtime object.

• One Orchestrator coordinates specialized capabilities.

• Agent ≠ Executor.

• One Model Gateway abstracts local/API models.

• Evidence-first execution and auditable outcomes.

• Quality gates before unrestricted AI-generated execution.

• Real-time events for execution, RBAC and API telemetry.

• Deterministic normalized test definitions.

• Human review for high-risk Security/RBAC/Business Logic work.

• Fault tolerance: agent/worker failure does not unnecessarily terminate unrelated work.

• Selective reporting reconciles with durable state. ■filecite■turn6file1■turn5file2■

16. Version Status

v1.0: Original Business Logic & Architecture baseline. **v2.0:** Updated with v6.0 Hybrid Multi-Agent architecture, Central Orchestrator, specialized intelligence, Test Case Validator, Execution Orchestrator/Workers, AI Model Gateway and Agent≠Executor separation. ■filecite■turn6file0■turn6file1■